

Node.js Advanced MCQ Questions - Situation Based

EventEmitter

Q1. You're building a real-time notification system. After adding 15 listeners to a single event, you notice a warning. What's happening and how do you fix it?

```
const emitter = new EventEmitter();
for(let i = 0; i < 15; i++) {
  emitter.on('notification', handler);
}
```

- A) EventEmitter has a default max listener limit of 10; use emitter.setMaxListeners(20)
- B) EventEmitter doesn't support more than 10 listeners; refactor to use multiple emitters
- C) Memory leak detected; use once() instead of on()
- D) No issue; warning can be ignored

Answer: A

Q2. Your application crashes when an error occurs in an EventEmitter. What's the issue?

```
const emitter = new EventEmitter();
emitter.emit('error', new Error('Something failed'));
```

- A) emit() doesn't support Error objects
- B) 'error' events without listeners throw exceptions; add error listener
- C) EventEmitter is not initialized properly
- D) Need to use try-catch around emit()

Answer: B

Q3. You need to ensure event listeners execute in a specific order and some should only run once. What's the best approach?

- A) Use on() for all and manually track execution count
- B) Use prependListener() for priority and once() for single execution
- C) EventEmitter doesn't guarantee order; use promises
- D) Create separate EventEmitters for each priority level

Answer: B

Q4. Memory leak detected in your event-driven application. What's the likely cause?

```
setInterval(() => {  
  const emitter = getGlobalEmitter();  
  emitter.on('data', processData);  
}, 1000);
```

- A) setInterval creates memory leaks by default
- B) Listeners are added repeatedly without removal; use once() or removeListener()
- C) EventEmitter can't be used with setInterval
- D) processData function is not garbage collected

Answer: B

Q5. You're implementing a pub-sub system. How do you pass multiple arguments to event listeners efficiently?

```
emitter.emit('userAction', userId, action, timestamp, metadata);
```

- A) This is correct; emit() supports multiple arguments
- B) Use emit('userAction', [userId, action, timestamp, metadata])
- C) Use emit('userAction', {userId, action, timestamp, metadata})
- D) Both A and C are valid; C is better for maintainability

Answer: D

Buffer

Q6. You're processing uploaded images and notice memory issues. What's wrong with this code?

```
const imageBuffer = Buffer.allocUnsafe(1024 * 1024 * 10);
fs.readFile('image.jpg', (err, data) => {
  data.copy(imageBuffer);
});
```

- A) allocUnsafe() may contain sensitive old data; use alloc() or fill buffer
- B) Buffer size is too large; split into chunks
- C) readFile() doesn't work with buffers
- D) copy() method is deprecated

Answer: A

Q7. You need to convert binary data to base64 for API transmission. Which approach is correct?

```
const binaryData = fs.readFileSync('file.pdf');
```

- A) binaryData.toString('base64')
- B) Buffer.from(binaryData).toString('base64')
- C) Buffer.from(binaryData, 'binary').toString('base64')
- D) A is correct; readFileSync returns a Buffer

Answer: D

Q8. You're building a file upload service. How do you efficiently handle large file buffers?

- A) Load entire file into Buffer using Buffer.alloc(fileSize)
- B) Use streams instead of loading entire file into Buffer
- C) Split file into multiple small Buffers manually
- D) Use Buffer.concat() to merge chunks

Answer: B

Q9. Your application needs to compare two buffers for authentication tokens. What's the secure way?

```
const token1 = Buffer.from(userToken);  
const token2 = Buffer.from(storedToken);
```

- A) token1.toString() === token2.toString()
- B) token1.equals(token2)
- C) crypto.timingSafeEqual(token1, token2)
- D) Buffer.compare(token1, token2) === 0

Answer: C

Q10. You're parsing a binary protocol. How do you read a 32-bit integer from a buffer at offset 4?

```
const buffer = Buffer.from([0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x64]);
```

- A) buffer.readInt32BE(4)
- B) buffer.readInt32LE(4)
- C) buffer.slice(4, 8).readInt32BE(0)
- D) Both A and C work; A is more efficient

Answer: D

Stream

Q11. You're processing a 5GB log file. Your application crashes with "JavaScript heap out of memory". What's the solution?

```
const data = fs.readFileSync('huge-log.txt', 'utf8');
const lines = data.split('\n');
```

- A) Increase Node.js heap size with --max-old-space-size
- B) Use fs.createReadStream() with line-by-line processing
- C) Split file into smaller chunks manually
- D) Use worker threads to process in parallel

Answer: B

Q12. Your file copy operation using pipe() fails silently. What's missing?

```
const readStream = fs.createReadStream('source.txt');
const writeStream = fs.createWriteStream('dest.txt');
readStream.pipe(writeStream);
```

- A) Need to call end() on writeStream
- B) Missing error handlers on both streams
- C) pipe() is deprecated; use pipeline()
- D) Need to set encoding on readStream

Answer: B

Q13. You need to compress data before writing to file. Which stream type should you use?

```
const input = fs.createReadStream('input.txt');
const output = fs.createWriteStream('output.gz');
```

- A) Duplex stream
- B) Transform stream with zlib.createGzip()
- C) Writable stream with compression option
- D) Readable stream with transform option

Answer: B

Q14. Your stream pipeline has memory leaks. What's the best practice?

```
readable.pipe(transform1).pipe(transform2).pipe(writable);
```

- A) Manually call destroy() on each stream
- B) Use stream.pipeline() with callback for proper cleanup
- C) Use stream.finished() to detect completion
- D) Add 'close' event listeners on all streams

Answer: B

Q15. You're implementing backpressure handling. What happens when you ignore it?

```
readable.on('data', (chunk) => {  
  writable.write(chunk);  
});
```

- A) Data loss occurs
- B) Memory consumption increases; should check write() return value
- C) Stream automatically pauses
- D) No issue; Node.js handles it automatically

Answer: B

File System

Q16. Your API endpoint reads files synchronously and becomes unresponsive under load. What's the issue?

```
app.get('/download/:file', (req, res) => {  
  const data = fs.readFileSync(`./files/${req.params.file}`);  
  res.send(data);  
});
```

- A) readFileSync() blocks event loop; use fs.promises.readFile() or streams

- B) Need to increase server timeout
- C) File path is incorrect
- D) Should use res.download() instead

Answer: A

Q17. You need to safely check if a file exists before reading. What's the correct approach?

- A) Use fs.exists() then fs.readFile()
- B) Use fs.access() then fs.readFile()
- C) Directly use fs.readFile() and handle ENOENT error
- D) Use fs.stat() to check file existence

Answer: C

Q18. Your application has a race condition when multiple processes write to the same file. How do you fix it?

```
fs.writeFile('counter.txt', count.toString(), callback);
```

- A) Use fs.writeFileSync() for atomic writes
- B) Use fs.open() with 'wx' flag for exclusive write
- C) Implement file locking mechanism or use database
- D) Add setTimeout() between writes

Answer: C

Q19. You're building a log rotation system. How do you append to a file efficiently for high-frequency writes?

- A) Use fs.appendFile() for each log entry
- B) Use fs.createWriteStream() with {flags: 'a'} and reuse stream
- C) Read file, append in memory, write back
- D) Use fs.writeFile() with append flag

Answer: B

Q20. Your file deletion fails with EBUSY error on Windows. What's the likely cause and solution?

```
fs.readFile('temp.txt', (err, data) => {  
  fs.unlink('temp.txt', (err) => {  
    // EBUSY error  
  });  
});
```

- A) File is locked by antivirus; retry with delay
- B) File handle not closed; use fs.close() or wait for GC
- C) Permission issue; run with elevated privileges
- D) Use fs.rm() instead of fs.unlink()

Answer: B

Folders/Directories

Q21. You need to create a nested directory structure. What's the issue with this code?

```
fs.mkdir('./uploads/users/avatars', (err) => {  
  // Error: ENOENT, parent directory doesn't exist  
});
```

- A) Use fs.mkdirSync() instead
- B) Create each directory level separately
- C) Use {recursive: true} option
- D) Both B and C work; C is better

Answer: D

Q22. You're building a file explorer. How do you recursively get all files in a directory tree?

- A) Use fs.readdir() with recursive option
- B) Use fs.readdir() with {withFileTypes: true} and recursively process
- C) Use fs.readdirSync() in a loop
- D) Use glob pattern matching

Answer: B

Q23. Your directory deletion fails because it's not empty. What's the solution?

```
fs.rmdir('./temp', (err) => {  
  // Error: ENOTEMPTY  
});
```

- A) Delete all files first, then directory
- B) Use fs.rm() with {recursive: true, force: true}
- C) Use fs.rmdirSync() instead
- D) Use fs.unlink() on directory

Answer: B

Q24. You need to watch a directory for new files and process them. What's the best approach?

```
fs.watch('./uploads', (eventType, filename) => {  
  // Process file  
});
```

- A) This is correct; fs.watch() is reliable
- B) fs.watch() is unreliable; use fs.watchFile() or chokidar library
- C) Use setInterval() to poll directory
- D) Use fs.stat() to check modification time

Answer: B

Q25. You're implementing a file cleanup service. How do you safely delete old files from a directory?

```
const files = fs.readdirSync('./temp');
files.forEach(file => fs.unlinkSync(file));
```

- A) This is correct
- B) Need full path: fs.unlinkSync(path.join('./temp', file))
- C) Use fs.rm() instead of fs.unlink()
- D) Check if file is directory first using fs.statSync()

Answer: B and D (Best: B with D check)

Mixed Advanced Scenarios

Q26. You're building a CSV parser for large files. What's the optimal architecture?

- A) fs.readFileSync() → split by lines → parse each line
- B) fs.createReadStream() → Transform stream → parse chunks
- C) fs.readFile() → Promise.all() for parallel parsing
- D) Worker threads with fs.readFileSync()

Answer: B

Q27. Your microservice needs to process uploaded files and emit progress events. What's the best design?

```
class FileProcessor extends EventEmitter {
  async process(filePath) {
    const stream = fs.createReadStream(filePath);
    // How to emit progress?
  }
}
```

- A) Track stream 'data' events and emit progress based on bytes read
- B) Use fs.stat() to get file size, calculate percentage in 'data' handler

- C) Use Transform stream to track progress and emit events
- D) Both B and C; C is more modular

Answer: D

Q28. You need to implement atomic file writes to prevent corruption. What's the approach?

- A) Use fs.writeFileSync() for atomic writes
- B) Write to temp file, then fs.rename() to target (atomic on POSIX)
- C) Use file locking with fs.open() and exclusive flags
- D) Use database instead of files

Answer: B

Q29. Your application processes binary protocol data over TCP. What's the correct approach?

```
socket.on('data', (chunk) => {  
  const header = chunk.readUInt32BE(0);  
  // Process message  
});
```

- A) This is correct
- B) Buffer chunks; messages may span multiple 'data' events
- C) Use socket.setEncoding('utf8')
- D) Convert to string first

Answer: B

Q30. You're implementing a file-based queue system. What's the challenge and solution?

```
const files = fs.readdirSync('./queue');  
files.forEach(file => processJob(file));
```

- A) Race condition with multiple workers; use fs.rename() for atomic claim
- B) No issue; filesystem operations are atomic

- C) Use `fs.watch()` to detect new files
- D) Add file locking mechanism

Answer: A